

Mirna Jeannette Reyes Jiménez

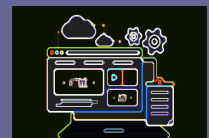
Junio 2025



02 - Modelos de Consistencia de Memoria.

Materia: computación paralela

Carrera: Tecnologías de Software



Práctica 02 - Modelos de Consistencia de Memoria.

Suban dos códigos de ejemplo en lenguaje Java, donde cada uno represente la consistencia de memoria Weak/Strong.

Pueden elegir el ejemplo que consideren más adecuado, siempre y cuando refleje correctamente los conceptos vistos en clase. Además, deberán elaborar un documento en formato PDF que funcione como documentación, donde expliquen cada uno de los cuatro ejemplos implementados, describiendo su funcionamiento y cómo se relaciona con el modelo correspondiente.

Puntos Extras.

Cargar archivos .c a él repositorio en GitHub utilizando tu rama personal.

Entregables:

- 2 archivos en Java (.java), cada uno representando una de las variantes de los modelos de consistencia de memoria.
- 1 documento en PDF, donde se explique detalladamente cada ejemplo y su relación.

Conceptos clave que reflejarán los códigos:

- **Consistencia Fuerte (Strong Consistency):** En este modelo, todas las escrituras son inmediatamente visibles para todas las lecturas. El orden de las operaciones es crucial y se garantiza que todos los observadores vean los eventos en el mismo orden global. Se logra a menudo con mecanismos de sincronización como `synchronized`, `volatile`, o `java.util.concurrent.atomic` en Java.
- **Consistencia Débil (Weak Consistency):** En este modelo, las escrituras no son necesariamente visibles de inmediato para todos los observadores. Puede haber un retraso o un orden diferente en que las actualizaciones son vistas por los distintos hilos. Esto puede llevar a que diferentes hilos vean estados de la memoria inconsistentes temporalmente. No se necesita una sincronización tan estricta, lo que a menudo permite mayor rendimiento pero requiere un manejo cuidadoso.

Archivo Java para Consistencia Fuerte (Strong Consistency)

Este modelo garantiza que todas las escrituras son inmediatamente visibles para todas las lecturas, manteniendo un orden global de operaciones. En Java, esto se logra comúnmente con la palabra clave `volatile` o bloques `synchronized`.

Este programa Java demuestra la consistencia fuerte mediante el uso de la palabra clave `volatile`. El objetivo es garantizar que un hilo lector siempre vea el valor actualizado de una variable después de que un hilo escritor la haya modificado y haya indicado que los datos están "listos".

El programa define una clase `StrongConsistencyExample` con dos variables estáticas: `ready` (un booleano) y `number` (un entero). Ambas son accesibles por múltiples hilos. La clave para la consistencia fuerte aquí es la declaración de `ready` como `volatile`.

- **WriterThread (Hilo Escritor):**

Este hilo inicializa la variable `number` con el valor 42.

Inmediatamente después, establece la variable `ready` a `true`.

Debido a que `ready` es `volatile`, cualquier escritura en `ready` crea una "barrera de memoria" (o un "happens-before relationship"). Esto garantiza que todas las escrituras realizadas *antes* de que `ready` se vuelva `true` (incluida la escritura de `number = 42`) serán visibles para cualquier hilo que lea `ready` y la encuentre como `true`.

- **ReaderThread (Hilo Lector):**

Este hilo entra en un bucle que espera activamente (`while (!ready)`) hasta que la variable `ready` se vuelve `true`.

Una vez que `ready` es `true`, el hilo lector sale del bucle e intenta leer el valor de `number`.

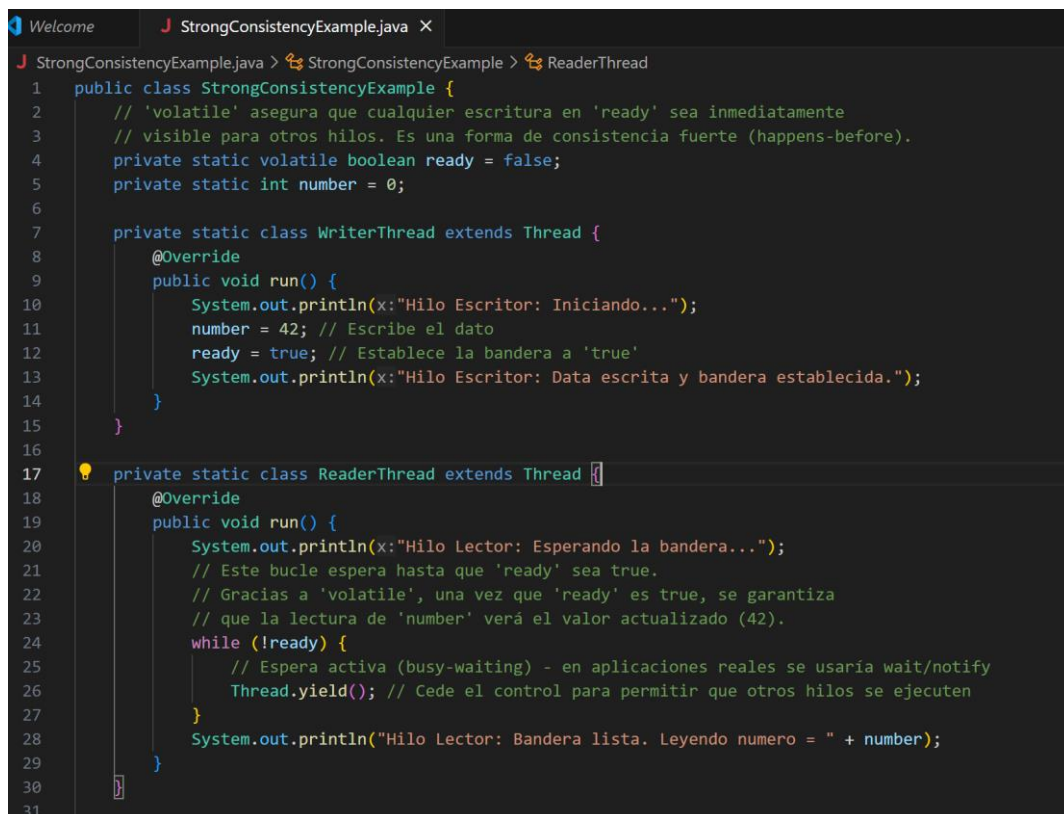
Gracias a la propiedad `volatile` de `ready`, se garantiza que cuando el hilo lector finalmente sale del bucle (porque `ready` es `true`), también verá el valor 42 para `number`, ya que la escritura de `number` por el hilo escritor ocurrió antes de que `ready` se estableciera.

- **main (Programa Principal):**

Crea y arranca una instancia de `ReaderThread` y otra de `WriterThread`.

El lector se inicia primero para simular que está esperando la señal del escritor.

Se utilizan `join()` para asegurar que el programa principal espere a que ambos hilos terminen su ejecución.



```
1 public class StrongConsistencyExample {
2     // 'volatile' asegura que cualquier escritura en 'ready' sea inmediatamente
3     // visible para otros hilos. Es una forma de consistencia fuerte (happens-before).
4     private static volatile boolean ready = false;
5     private static int number = 0;
6
7     private static class WriterThread extends Thread {
8         @Override
9         public void run() {
10             System.out.println(x:"Hilo Escritor: Iniciando...");
11             number = 42; // Escribe el dato
12             ready = true; // Establece la bandera a 'true'
13             System.out.println(x:"Hilo Escritor: Data escrita y bandera establecida.");
14         }
15     }
16
17     private static class ReaderThread extends Thread {
18         @Override
19         public void run() {
20             System.out.println(x:"Hilo Lector: Esperando la bandera...");
21             // Este bucle espera hasta que 'ready' sea true.
22             // Gracias a 'volatile', una vez que 'ready' es true, se garantiza
23             // que la lectura de 'number' verá el valor actualizado (42).
24             while (!ready) {
25                 // Espera activa (busy-waiting) - en aplicaciones reales se usaría wait/notify
26                 Thread.yield(); // Cede el control para permitir que otros hilos se ejecuten
27             }
28             System.out.println("Hilo Lector: Bandera lista. Leyendo numero = " + number);
29         }
30     }
31 }
```

Run | Debug

```
32 public static void main(String[] args) {
33     // Crea y arranca los hilos
34     ReaderThread reader = new ReaderThread();
35     WriterThread writer = new WriterThread();
36
37     reader.start(); // El lector se inicia primero para esperar al escritor
38     writer.start();
39
40     try {
41         reader.join(); // Espera a que el hilo lector termine
42         writer.join(); // Espera a que el hilo escritor termine
43     } catch (InterruptedException e) {
44         e.printStackTrace();
45     }
46
47     System.out.println(x:"Programa principal terminado.");
48 }
49 }
```

```
MINGW64:/c:/Users/mirna/OneDrive/Desktop/repositorio Mirna/Ceti-PP-202...
mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeannetteReyesJimenez)
$ cd Practica02_ConsistenciaMemoria

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025/Practica02_ConsistenciaMemoria (MirnaJeannetteReyesJimenez)
$ javac StrongConsistencyExample.java

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025/Practica02_ConsistenciaMemoria (MirnaJeannetteReyesJimenez)
$ java StrongConsistencyExample
Hilo Lector: Esperando la bandera...
Hilo Escritor: Iniciando...
Hilo Escritor: Data escrita y bandera establecida.
Hilo Lector: Bandera lista. Leyendo numero = 42
Programa principal terminado.

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025/Practica02_ConsistenciaMemoria (MirnaJeannetteReyesJimenez)
$ ^C

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025/Practica02_ConsistenciaMemoria (MirnaJeannetteReyesJimenez)
$
```

Archivo Java para Consistencia Débil (Weak Consistency)

En este ejemplo, se va a simular un escenario donde la visibilidad de los cambios entre hilos no está garantizada inmediatamente. Haremos esto eliminando `volatile` y agregando un pequeño retraso para aumentar la probabilidad de que el lector lea un valor "antiguo" o inesperado.

La consistencia débil es un modelo de consistencia más relajado que el fuerte. En este modelo, las escrituras realizadas por un hilo no están garantizadas de ser inmediatamente visibles para otros hilos. Puede haber un retraso antes de que las actualizaciones de memoria se propaguen por completo a todos los observadores. Esto puede llevar a situaciones donde diferentes hilos vean estados de memoria inconsistentes temporalmente, o vean las operaciones en un orden diferente. La consistencia débil permite un mayor rendimiento al reducir la sobrecarga de sincronización, pero requiere que el programador gestione explícitamente la sincronización cuando la visibilidad inmediata sea crítica.

Similar al ejemplo de consistencia fuerte, este programa también define `ready` y `number`. La diferencia crucial es que ninguna de estas variables es `volatile`.

- **WriterThread (Hilo Escritor):**

Escribe `number = 42` y luego `ready = true`.

Se ha introducido un pequeño `Thread.sleep(10)` entre la escritura de `number` y `ready` para aumentar la probabilidad de observar un comportamiento inconsistente, aunque no lo garantiza en todos los sistemas o ejecuciones.

- **ReaderThread (Hilo Lector):**

Espera en un bucle (`while (!ready)`) hasta que `ready` se vuelve `true`.

Debido a la ausencia de `volatile`, no hay una "garantía happens-before" entre la escritura de `number` y la lectura de `ready`. Esto significa que el hilo lector puede ver `ready` como `true` (porque el valor ha sido eventualmente escrito en la memoria principal), pero su caché local aún podría tener el valor antiguo de `number` (que es 0).

- **main (Programa Principal):**

La lógica es similar a la del ejemplo fuerte, creando y arrancando los hilos, y esperando su finalización.

J WeakConsistencyExample.java > WeakConsistencyExample

```
1 public class WeakConsistencyExample {
2     // En este ejemplo, 'ready' y 'number' NO son 'volatile'.
3     // Esto significa que los cambios hechos por un hilo NO están garantizados
4     // de ser visibles inmediatamente para otros hilos en el orden esperado.
5     private static boolean ready = false; // Sin volatile
6     private static int number = 0;        // Sin volatile
7
8     private static class WriterThread extends Thread {
9         @Override
10        public void run() {
11            System.out.println(x:"Hilo Escritor: Iniciando...");
12            number = 42; // Escribe el dato
13            // Añadimos un pequeño retraso para aumentar la probabilidad de inconsistencia
14            try {
15                Thread.sleep(millis:10); // 10 milisegundos
16            } catch (InterruptedException e) {
17                Thread.currentThread().interrupt();
18            }
19            ready = true; // Establece la bandera
20            System.out.println(x:"Hilo Escritor: Data escrita y bandera establecida.");
21        }
22    }
23
24    private static class ReaderThread extends Thread {
25        @Override
26        public void run() {
27            System.out.println(x:"Hilo Lector: Esperando la bandera...");
28            // Este bucle espera hasta que 'ready' sea true.
29            // Sin 'volatile', NO hay garantía de que, cuando 'ready' se haga true,
30            // 'number' ya haya sido visto como 42. Podría seguir siendo 0 en el caché del lector.
31            while (!ready) {
32                // Espera activa. En un sistema débilmente consistente, 'ready' puede hacerse
33                // true sin que 'number' se sincronice aún.
34                Thread.yield();
35            }
36            System.out.println("Hilo Lector: Bandera lista. Leyendo numero = " + number);
37        }
38    }
39 }
```

```
39
40 Run | Debug
41 public static void main(String[] args) {
42     ReaderThread reader = new ReaderThread();
43     WriterThread writer = new WriterThread();
44
45     reader.start(); // El lector se inicia primero
46     writer.start();
47
48     try {
49         reader.join();
50         writer.join();
51     } catch (InterruptedException e) {
52         e.printStackTrace();
53     }
54
55     System.out.println(x:"Programa principal terminado.");
56     System.out.println(x:"Nota: Debido a la Consistencia Débil, el Hilo Lector PUEDE haber leído '0' para 'number'");
57     System.out.println(x:"Ejecute varias veces para observar la inconsistencia (puede variar según el sistema).");
58 }
59
```

```
MINGW64:/c/Users/mirna/OneDrive/Desktop/repositorio Mirna/Ceti-PP-202...
mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025/Practica02
_ConsistenciaMemoria (MirnaJeannetteReyesJimenez)
$ javac WeakConsistencyExample.java

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025/Practica02
_ConsistenciaMemoria (MirnaJeannetteReyesJimenez)
$ java WeakConsistencyExample
Hilo Lector: Esperando la bandera...
Hilo Escritor: Iniciando...
Hilo Escritor: Data escrita y bandera establecida.
Hilo Lector: Bandera lista. Leyendo numero = 42
Programa principal terminado.
Nota: Debido a la Consistencia Débil, el Hilo Lector PUEDE haber leído '0' para
'number' si no hay sincronización.
Ejecute varias veces para observar la inconsistencia (puede variar según el sist
ema).

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025/Practica02
_ConsistenciaMemoria (MirnaJeannetteReyesJimenez)
$ ^C

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025/Practica02
_ConsistenciaMemoria (MirnaJeannetteReyesJimenez)
$
```



Conclusión

Los modelos de consistencia de memoria son un pilar fundamental en la programación concurrente y paralela. La **consistencia fuerte** ofrece la simplicidad de un modelo de programación secuencial donde todos los cambios son inmediatamente visibles, a costa de una posible menor eficiencia debido a la sobrecarga de sincronización (como el uso de `volatile` o `synchronized`). Por otro lado, la **consistencia débil** prioriza el rendimiento al relajar las garantías de visibilidad, pero exige un diseño más cuidadoso y una sincronización explícita por parte del programador para evitar errores lógicos. La elección del modelo adecuado depende de los requisitos específicos de la aplicación en cuanto a rendimiento y la necesidad de garantías de orden y visibilidad de los datos.